Politecnico di Milano
School of Industrial and Information Engineering

Jan Wangrat

Intelligent Job Management

Final Report
COMPUTER SCIENCE AND ENGINEERING

Supervisor:
Prof. Danilo Ardagna
Politecnico di Milano
with Federica Filippini, Ph.D. Candidate

Milano, May 28, 2026

Abstract

Intelligent Job Management (IJM) is a scheduler for shared GPU clusters that combines profile-aware placement, deadline-driven preemption, and cross-version checkpoint resume. It extends the ANDREAS project (Politecnico di Milano) from a single-host prototype to a distributed control plane: one FastAPI service drives per-node workers over SSH-tunnelled HTTP, an external GPUspb optimiser handles standard-run placement, and training jobs migrate between heterogeneous GPU generations (A40, QuadroP600) without losing progress. The report describes the deployment topology, the four-phase scheduling pipeline, the configuration surface, and two end-to-end scenarios that exercise the system on a two-node cluster.

Keywords

GPU scheduling, deep learning, job management, profile-aware optimisation, distributed systems, FastAPI, PostgreSQL, Docker

Contents

1. Introduction

Modern deep-learning research depends on shared GPU clusters in which training jobs compete for a small number of expensive accelerators. Operators are expected to place each job on the hardware that meets its deadline at the lowest cost, despite per-GPU prices that vary by more than an order of magnitude between generations. Manual scheduling quickly becomes the bottleneck, and off-the-shelf cluster managers (Kubernetes, plain SLURM) treat GPU bundles as interchangeable and have no notion of per-job energy budgets or deadline-aware preemption.

The ANDREAS project [AF+21] at Politecnico di Milano demonstrated that a cost-aware optimiser, fed with profile measurements of each job type on each hardware bundle, can plan placements that beat the typical operator's intuition. ANDREAS, however, is a single-host prototype: it runs every container on the machine that hosts the API, has no remote worker, and assumes one homogeneous GPU pool. Real laboratories — the two-node MIM UW / Polimi cluster used in this work being a representative example — run jobs across nodes with different driver generations, must resume preempted training across CUDA versions, and want to consume a standalone optimiser as an HTTP service rather than ship it inside the scheduler.

This report describes Intelligent Job Management (IJM), a scheduler that extends the ANDREAS model to a distributed cluster. The control plane is a single FastAPI service that hosts the profiling scheduler, the optimiser client, and the per-node slot semaphores. Each GPU node runs a lightweight worker that owns the local Docker daemon. The two are glued together by plain HTTP over SSH tunnels, so the same code runs on a laptop talking to a remote cluster and on a single-host development machine. IJM integrates the GPUspb optimiser as the external decision maker for standard-run placement. Before any instance is placed as a standard run, the scheduler first executes a small number of profile runs for it on previously-unmeasured (node, GPU-count) cells, so the optimiser is never asked to score an unmeasured bundle. Checkpoints are written in a format that loads across torch versions, allowing a job to migrate between an A40 node and a legacy P600 node mid-training.

We validate the design with two end-to-end scenarios on the two-node cluster. Scenario 1 submits five `cnn_big` jobs to probe the profile sweep and the deadline-driven URGENT migration sequence. Scenario 2 mixes two job types and constructs a workload that pits URGENT against priority-5 pins on A40 and slack lstm-small patients on P600. Both scenarios expose the same horizon-myopia behaviour in the upstream GPUspb proxy.

Chapter 2 describes the deployment topology, the scheduling pipeline, the state machine, the persistent state, and the invariants that protect the control plane from concurrency hazards. Chapter 3 documents the cluster and job configuration files, the submission API, and the runtime knobs. Chapter 4 presents the two scenarios, the cost model used by the optimiser, and the implementation notes accumulated while bringing the scenarios to a reproducible state. Chapter 5 summarises the contributions, the limitations, and the follow-up work that has concrete entry points in the codebase.

2. System Architecture

This section presents IJM's deployment topology, the four-phase scheduling pipeline that drives every dispatch decision, the job state machine, the persistent state held in PostgreSQL, and the concurrency invariants that keep the control plane consistent under racing API calls and worker notifications. Table 2.1 catalogues every component referenced in the rest of the chapter.

Component	Process	Responsibility
API	`ijm-api` (FastAPI)	HTTP frontend; hosts the scheduling and dispatch loops
`ProfilingScheduler`	in-API	Picks the next unmeasured `(node, GPU-count)` cell for profile-owing rows
`JobDispatcher`	in-API	Acquires the destination's `NodeSlots` permit, then POSTs `/jobs/{id}/run` to the target worker
`NodeSlots`	in-API	Per-node semaphore tracking GPU permits; reconciled with the DB by a periodic drift heartbeat
`ClusterManager`	in-API	Loads `nodes_config.json` and exposes node and GPU resources
Optimizer client	in-API	Packages the GPUspb request, parses the returned assignments and preempts
Worker	`ijm-worker` (FastAPI)	Owns the local Docker daemon; performs atomic claim, runs the container, emits slot-freed notifications
PostgreSQL	DB	Source of truth for job rows and profile measurements
GPUspb optimiser	external HTTP	Cost-aware standard-run placement (reached via `OPTIMIZER_URL`)
SPA	static (Vite)	Operator UI; polls the API every 3–5 s

Table 2.1: IJM components and their responsibilities.

2.1 Deployment topology

IJM runs as a small distributed system. The FastAPI *API* — which also hosts the `ProfilingScheduler`, the `JobDispatcher`, the `NodeSlots` semaphores, and the optimiser client — runs on the operator's machine. A lightweight FastAPI *worker* runs on every GPU node and owns the local Docker daemon. PostgreSQL runs on one of the GPU nodes.

Communication is plain HTTP over SSH tunnels. Every worker is reached from the API at a stable `http://localhost:800x` alias regardless of its physical location, so the same scheduler code drives a laptop talking to a remote cluster and a single-host development setup. An optional GPUspb *optimiser* HTTP service is reached the same way.

Single-process `docker compose` on the operator's machine is supported for development but is not the deployment target: any tradeoff between local-dev simplicity and distributed correctness resolves toward the distributed topology.

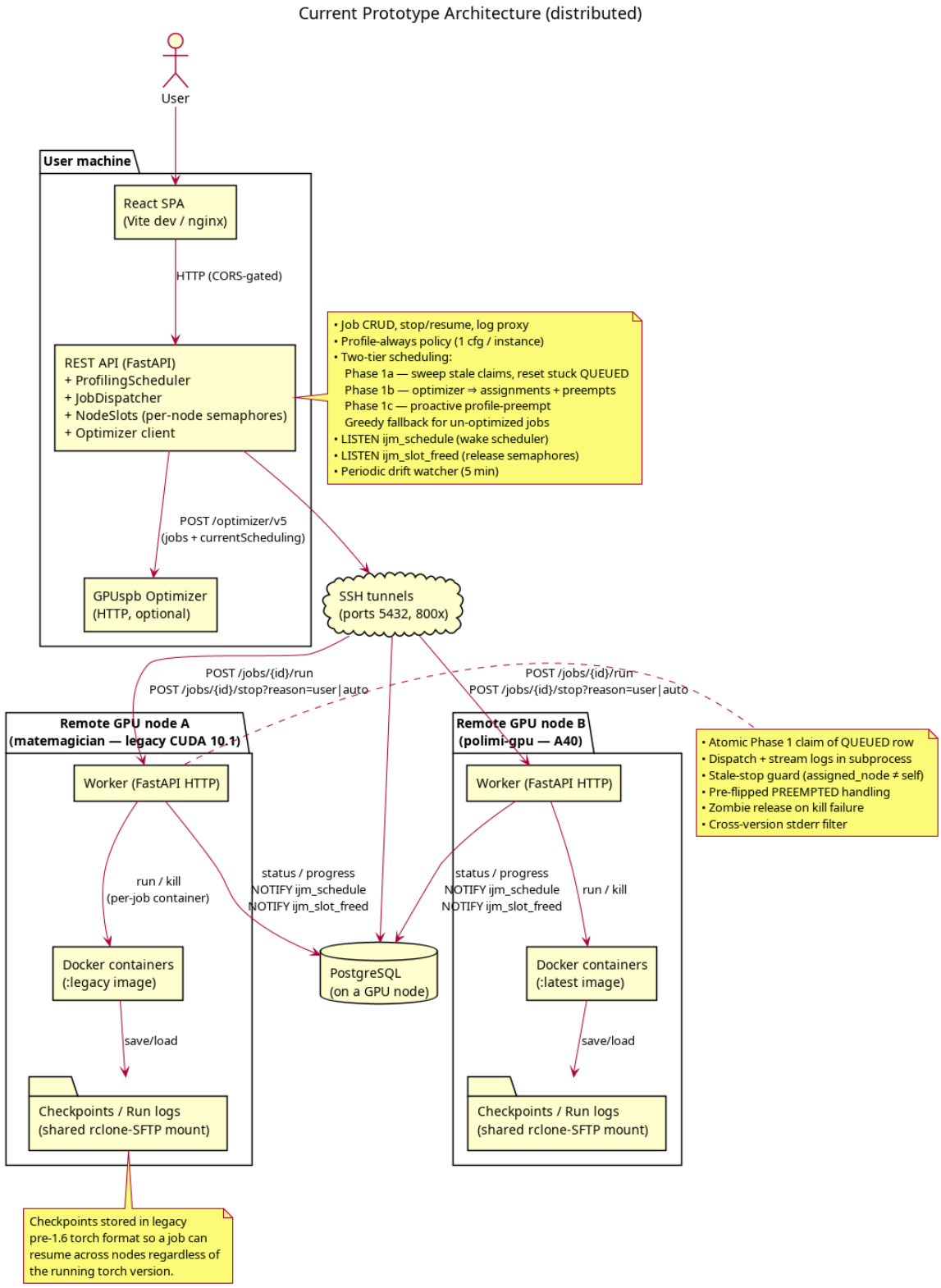

Figure 2.1: Distributed prototype. The SPA, API, and optimiser run on the operator's machine; workers and Docker run on each GPU node; PostgreSQL runs on a GPU node. A shared rclone-over-SFTP mount makes checkpoints written by one node visible from the other.

2.2 Scheduling pipeline

A single scheduling pass — `_schedule_waiting_jobs` in `app.py` — handles every dispatch decision. It is invoked whenever a job is submitted, a profiling run finishes, or a slot is freed. A 60-minute watchdog (`_queue_watcher`) serves as a safety net for missed notifications, not as the main wake source. Wake-ups are coalesced through a single `asyncio.Event` and rate-limited to at most one pass every five seconds, so the cluster has time to apply the previous plan before the next one is computed.

The pass walks four phases under a single `schedule_lock`:

1. **Reset stuck assignments.** Reset `QUEUED` rows whose `assigned_node` was set ≥ 30 s ago without the worker advancing them, so a later phase can re-place the row.
2. **Optimiser pass.** Call the optimiser outside the lock (to accommodate its 30 s HTTP timeout) and apply its assignments through a conditional `UPDATE ...WHERE assigned_node IS NULL`, collecting the optimiser's preempt list. The optimiser only ever scores rows with `is_profiling_run = FALSE` — rows still owing a profile run are invisible to it.
3. **Profile placement.** For `QUEUED` rows that still owe a profile run, the `ProfilingScheduler` atomically claims one unmeasured (node, GPU-count) cell using a best-fit-on-remaining-capacity rule. This is the mechanism behind the *profile-always* policy: an instance must complete `PROFILING_CONFIGS_PER_JOB` profile runs (default 2) before it becomes a candidate for the optimiser-driven standard placement above.
4. **Profile-preempt.** If a profile-owing row still has no slot after the placement pass, find the cheapest set of low-priority `RUNNING` jobs whose eviction would free enough GPUs for the next unmeasured configuration.

Outside the lock the pass spawns one task per preempt and one per dispatch. Each dispatch goes through `JobDispatcher.dispatch_with_slot`, which acquires the destination's `NodeSlots` permit before sending `POST /jobs/{id}/run` to the right worker. Permits are released when the worker emits a `NOTIFY ijm_slot_freed` payload of the form `<node_id>:<n_gpus>:<reason>`. The listener always releases the permit; whether it also wakes the optimiser is gated by the reason. `terminal` (job ended naturally), `user_stop` (UI-initiated), and `orphan_drain` (zombie cleanup) are external events and trigger an optimiser pass. `auto_preempt` is the API's own `/stop` issued as part of an in-flight plan; the plan's own dispatch step will pick up the just-released slot, so re-running the optimiser there would thrash the plan it is already executing. The reason is set at the worker's emit site (`SlotFreedReason` in `shared/constants.py`).

Figure 2.2 shows the submission path, Figure 2.3 the cross-instance profile cycle, Figure 2.4 an optimiser-driven preempt and migrate round, and Figure 2.5 the standard run.

2.3 State machine

Table 2.2 lists every transition a job row can take. All transitions are driven by atomic conditional `UPDATE`s, so concurrent stop, dispatch, and completion paths cannot clobber each other's work.

Figure 2.2: Submission and scheduling. The API writes the row and notifies the scheduler; placement is decided by the next pass. The optimiser call sits outside the schedule lock so its 30 s HTTP timeout cannot block other passes.

From	To	Trigger
QUEUED	PROFILING / RUNNING	Worker Phase 1 atomic claim
PROFILING	QUEUED	Profile run finished; `is_profiling_run` flips to `false`
PROFILING	FAILED	Profile container exited non-zero
RUNNING	SUCCEEDED / FAILED	Container exit
RUNNING	PREEMPTED	User `POST /jobs/{id}/stop?reason=user`
RUNNING	QUEUED	Scheduler preempt (`reason=auto`): clears `assigned_node` for the next pass
PREEMPTED / FAILED	QUEUED	`POST /jobs/{id}/resume`

Table 2.2: Job state transitions.

Two distinct stop reasons separate *user-initiated* preemption from *scheduler-initiated* preemption. A user stop is sticky: PREEMPTED stays put until the user explicitly resumes. A scheduler stop returns the row to QUEUED with a cleared assignment, so the next pass can place it elsewhere automatically. The worker enforces this distinction and ignores stale `/stop` calls aimed at a row that has since been re-assigned to another node.

2.4 Persistent state

Two PostgreSQL tables hold every fact the scheduler reads or writes; both are created idempotently on startup from `backend/schema.sql`.

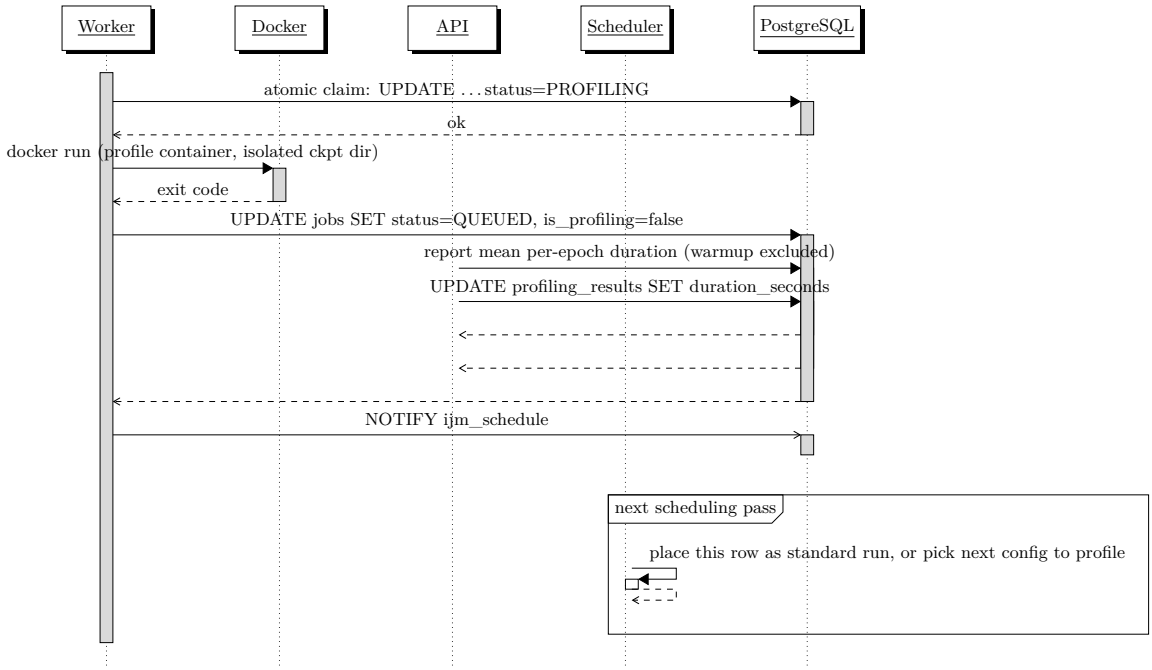

Figure 2.3: Profiling cycle. Profile measurements are keyed by the (job-type, GPU-config) pair and shared across submissions of the same type. Each instance contributes one profile run by default; the per-instance claim row guards against duplicate measurements.

```
CREATE TABLE jobs (
    id                    TEXT PRIMARY KEY,
    job_id                TEXT NOT NULL,
    image                 TEXT NOT NULL,
    command               JSONB NOT NULL,
    script_path           TEXT,
    directory_to_mount    TEXT,
    status                TEXT NOT NULL,
    created_at            TIMESTAMPTZ NOT NULL,
    updated_at            TIMESTAMPTZ NOT NULL,
    container_name        TEXT,
    exit_code             INT,
    progress              TEXT,
    priority              INT DEFAULT 3,
    deadline              TIMESTAMPTZ,
    batch_size            INT,
    epochs_total          INT,
    profiling_epochs_no   INT,
    assigned_node         TEXT,
    assigned_gpu_config   JSONB,
    is_profiling_run      BOOLEAN DEFAULT FALSE
);

CREATE TABLE profiling_results (
    id                TEXT PRIMARY KEY,
    job_id            TEXT NOT NULL,
    instance_id       TEXT,
    gpu_config        JSONB NOT NULL,
    node_id           TEXT NOT NULL,
    duration_seconds  FLOAT,
    created_at        TIMESTAMPTZ NOT NULL
);
```

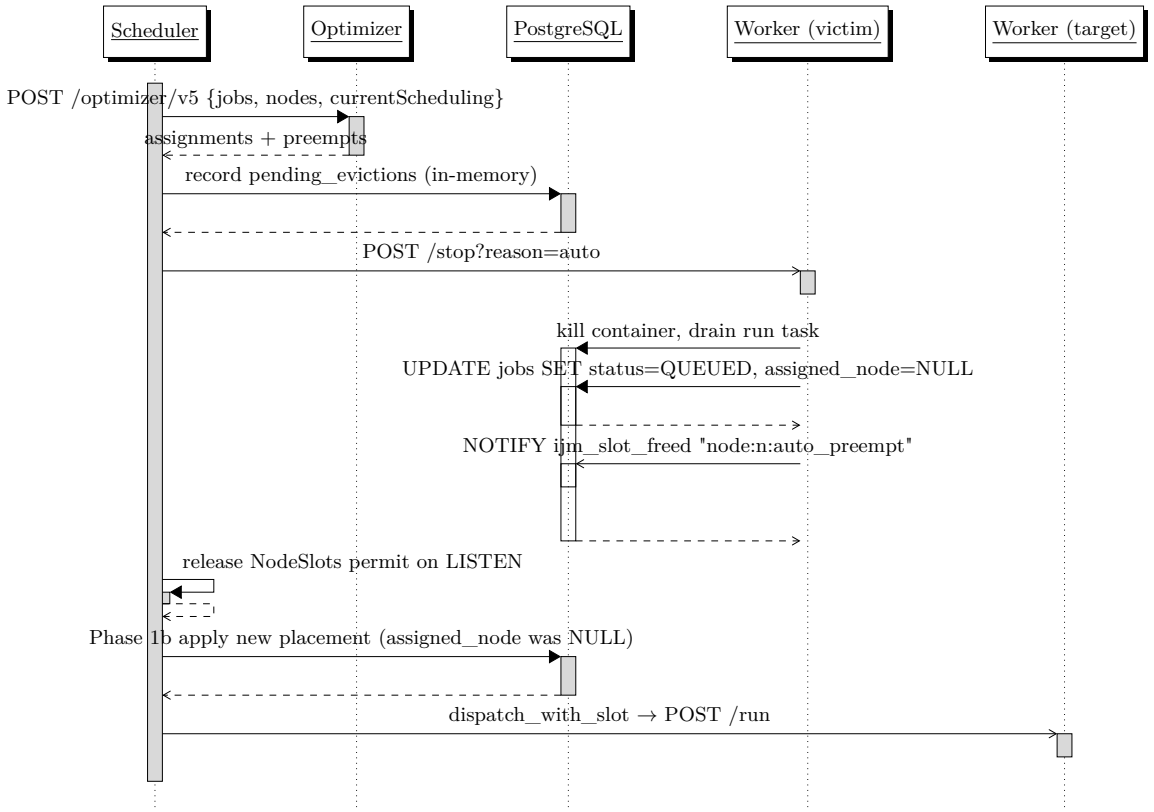

Figure 2.4: Optimiser-driven preempt and migrate. The API trusts the GPUspb plan: any job in `currentScheduling` that the optimiser drops or moves is preempted with `reason=auto`, which clears its assignment so Phase 1b can re-apply the new placement on the next pass.

Two columns of `jobs` are load-bearing for the scheduling pass above: `assigned_node` is the target the optimiser pass writes under `WHERE assigned_node IS NULL`, and `is_profiling_run` is the flag that makes a row invisible to the optimiser (Phase 1b) and visible to the `ProfilingScheduler` (Phase 1c).

A unique index on `(job_id, gpu_config)` makes the cross-instance profile claim atomic. Profile measurements are keyed by the *job type* (`job_id`) and the GPU configuration, so two concurrent submissions of the same type race to own each unmeasured cell; the loser's INSERT trips the unique constraint and the winner's measurement is shared by every subsequent instance of that type.

2.5 Concurrency invariants

The control plane relies on three invariants, each defended in code.

No GPU oversubscription. Every dispatch goes through `NodeSlots.acquire(node_id, n_gpus)`. On startup the semaphores are reconciled from the database; rows in RUNNING or PROFILING, and QUEUED rows with an assigned node, pre-acquire their permits. A periodic drift watcher recomputes the authoritative count at the `IJM_DRIFT_HEARTBEAT_S` cadence (default 15 s) and corrects any divergence.

No duplicate containers per row. Both the worker (Phase 1 claim, `running_jobs[job_id]` check) and the API (`dispatch_tasks[instance_id]` deduplication) refuse a second concurrent dispatch for the same instance. The reaper that resets stuck QUEUED rows cancels any live dispatch task instead of double-releasing its permit.

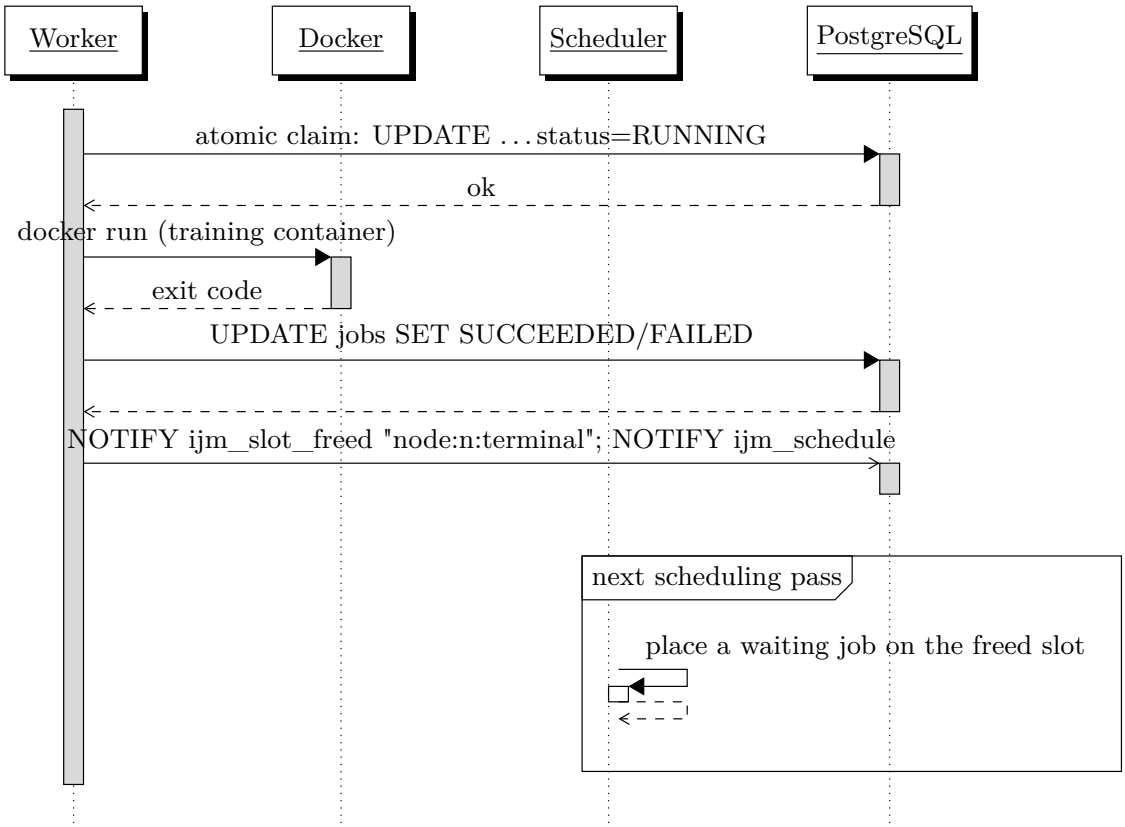

Figure 2.5: Standard run. The worker performs an atomic conditional `UPDATE ...WHERE status = ANY(RUNNABLE_STATUSES)` so concurrent stops or duplicate dispatches cannot race a row into `RUNNING`. On exit the worker fires `ijm_slot_freed` and `ijm_schedule` in the same transaction.

Profiling is sacred. Auto-preempt (`_preempt_and_release`) refuses to act on `is_profiling_run=TRUE`, and profile-preempt (Phase 1c) filters victims by `status='RUNNING'` only. Only an explicit user stop can cancel an in-flight measurement.

2.6 Cross-node checkpoints and image variants

Two GPU generations coexist. The `matemagician` node is pinned to driver 418.x (CUDA 10.1 max), so it runs a separate `:legacy` image built on PyTorch 1.5.1 and Python 3.7. The remaining nodes run the default `:latest` image on PyTorch 2.6 and CUDA 12.4. Both images write checkpoints in the legacy pre-1.6 `torch.save` format, and both load with `strict=False`, so a job can resume across nodes regardless of which torch version produced the file.

The shared `data/` directory is mounted on every node via rclone over SFTP. It contains:

- `data/checkpoints/{job_id}/` — per-job checkpoints, mounted into the container at `/checkpoints`;
- `data/runs/{job_id}/` — captured stdout (`output.log`) and TensorBoard artifacts, mounted at `/runs`;
- `data/shared/data/` — read-only dataset cache (MNIST, CIFAR-10), mounted at `/runs/data`.

Two worker-side environment variables let the same `nodes_config.json` drive heterogeneous nodes. `WORKER_GPU_MODE` selects how Docker exposes GPUs to the training container: `runtime` for the standard NVIDIA runtime, `cdi` for the rootless daemon on `polimi-gpu`, and `none` for a CPU-only sandbox. `IMAGE_TAG_OVERRIDE` rewrites the image tag, which is how `matemagician` swaps `:latest` for `:legacy`.

3. Configuration and Job Submission

IJM is configured at three layers: a pair of JSON files that describe the cluster, an HTTP submission API for jobs, and a handful of runtime environment variables. This section documents each in turn.

3.1 Cluster configuration

The cluster topology is loaded at startup from two JSON files, whose paths are configurable via environment variables.

3.1.1 Nodes (`nodes_config.json`)

Each entry describes a compute node and its GPU resources. The example below is the deployment used throughout Chapter 4 (`config/nodes_config.tunnel.json`); the file checked in at `config/nodes_config.json` is a mock cluster used by the unit tests.

```
[
  {
    "id": "matemagician",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8001",
    "cost": 0.10,
    "resources": [
      { "gpu_type": "QuadroP600", "gpu_count": 2 }
    ]
  },
  {
    "id": "polimi-gpu",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8002",
    "cost": 0.30,
    "resources": [
      { "gpu_type": "A40", "gpu_count": 2 }
    ]
  }
]
```

Field	Type	Description
`id`	string	Unique node identifier
`isForProfiling`	bool	If `true`, the node can host profiling runs. Production jobs may still land on it; profiling jobs may only land on it
`workerUrl`	string | null	HTTP base URL of the worker, typically reached through an SSH tunnel. When `null`, dispatch falls through to the in-process `JobRunner` (development only)
`cost`	float	Idle cost of the node ($/h), used by the optimiser
`resources`	array	GPU resources attached to the node
`resources[].gpu_type`	string	GPU model name (e.g. `A40`, `L40S`, `QuadroP600`)
`resources[].gpu_count`	int	Number of GPUs of this type

Table 3.1: Node configuration fields.

GPU type and count are declared explicitly rather than auto-detected. This admits mixed-GPU nodes that ANDREAS did not support (e.g. a node with both A40 and L40S GPUs), and it lets the operator declare nodes that are not physically attached to the API host — the API only ever talks to workers over HTTP.

3.1.2 Energy costs (`gpu_energy_costs.json`)

A lookup table that maps a GPU type and bundle size to an hourly energy cost (EUR/h). The scheduler consults this table when scoring candidate placements.

```
{
  "A40":         { "1": 0.30, "2": 0.55, "4": 1.05 },
  "L40S":        { "1": 0.40, "2": 0.75, "4": 1.40 },
  "QuadroP600": { "1": 0.03, "2": 0.055 },
  "Blackwell":  { "1": 1.00, "2": 1.85, "4": 3.50 }
}
```

3.2 Job submission

Endpoint: `POST /jobs`

3.2.1 Request body

```
{
  "job_id": "LSTM-small",
  "dockerImage": "ijm-lstm-small:dev",
  "scriptPath": "train.py",
  "directoryToMount": "/data/my-project",
  "Priority": 3,
  "deadline": "2026-03-20T18:00:00",
  "batchSize": 64,
  "profilingEpochsNo": 3,
  "epochsTotal": 20
}
```

Field	Type	Required	Description
`job_id`	string	yes	Reusable job type identifier; profile measurements are correlated across submissions sharing the same `job_id`
`dockerImage`	string	yes	Docker image to execute
`scriptPath`	string	no	Path to the training script (overrides Docker CMD with `python3 <scriptPath>`)
`directoryToMount`	string	no	Host directory mounted into the container at `/workspace`
`command`	string[]	no	Command and arguments (defaults to the image entrypoint; overridden by `scriptPath`)
`Priority`	int 1–5	no	Job priority (default 3)
`deadline`	datetime	no	Job deadline (ISO 8601)
`batchSize`	int	no	Batch size passed as the `BATCH_SIZE` env var
`profilingEpochsNo`	int ≥ 1	no	Epochs per profiling run (default 3)
`epochsTotal`	int ≥ 1	no	Total training epochs (default 20)

Table 3.2: Job creation request fields.

Each submission receives a unique UUID instance ID (`id`) for lifecycle tracking, separate from the `job_id` type identifier. When `scriptPath` is provided the container runs `python3 <scriptPath>`; otherwise the image's default CMD is used. Unlike ANDREAS, `Priority` and `deadline` are optional, and an additional `command` field gives flexibility for non-standard workloads.

3.2.2 Response (201 Created)

The response carries the full job object, including the scheduler's initial decision: assigned node, GPU configuration, and whether the first run will be a profile or a standard execution.

```
{
  "id": "a1b2c3d4-...",
  "job_id": "LSTM-small",
  "image": "ijm-lstm-small:dev",
  "command": ["python3", "train.py"],
  "status": "QUEUED",
  "is_profiling_run": true,
  "assigned_node": "node-mixed-prof",
  "assigned_gpu_config": { "A40": 1 },
  "priority": 3,
  "epochs_total": 20,
  "profiling_epochs_no": 3,
  "created_at": "2026-03-18T10:00:00Z",
  "updated_at": "2026-03-18T10:00:00Z",
  ...
}
```

3.3 Full API overview

Table 3.3 lists every endpoint. Interactive documentation is available at `/docs` (Swagger UI) and `/redoc` when the API is running.

Method	Path	Description
GET	/	Health check endpoint
GET	/health	Detailed health check — pings DB and checks job runner
GET	/nodes	List all cluster nodes with their current status
GET	/gpu-costs	Hourly GPU energy costs used by the scheduler for optimisation
GET	/configurations	List all valid hardware configurations in the cluster
POST	/jobs	Create a new job
GET	/jobs	List all jobs ordered by creation time (paginated via limit/offset)
GET	/jobs/{id}	Get a specific job by ID
POST	/jobs/{id}/stop	Request job to be stopped (user-driven; sticky PREEMPTED state)
POST	/jobs/{id}/resume	Request job to be resumed
DELETE	/jobs/{id}	Delete a job, killing any running container first
DELETE	/jobs	Delete all jobs and their profiling results
GET	/jobs/{id}/logs	Return the output log for a job by proxying to the worker that owns it
GET	/profiling-results/{id}	All profiling results for a job, ordered by duration (fastest first)
GET	/admin/slots	NodeSlots snapshot: in-memory used vs. DB-authoritative count
POST	/admin/reconcile-slots	Force NodeSlots to match DB-authoritative state (idempotent)
GET	/admin/dispatch-tasks	List in-flight _dispatch_when_slot_free tasks

Table 3.3: API endpoints.

Three architectural choices set the API apart from ANDREAS. First, scheduling, profiling, and dispatch are consolidated into a single FastAPI service, in contrast to ANDREAS's split between a Java Jobs Manager and a C++ Jobs Optimiser. Second, execution lives in per-node *workers* — also FastAPI — that the API reaches over HTTP through SSH tunnels; the external GPUspb optimiser is reached the same way and owns every standard-run placement decision. Third, IJM exposes explicit `stop`, `resume`, `logs`, and `delete` endpoints, together with `/profiling-results` and `/configurations`, so the user has direct visibility into the scheduler's state.

3.4 Runtime environment

Table 3.4 lists the runtime knobs that influence the scheduler. Unless noted otherwise, every variable is read by the API process at startup.

Variable	Default	Description
DATABASE_URL	postgresql://.../ijm	PostgreSQL DSN, reached via SSH tunnel
OPTIMIZER_URL	unset	The GPUspb HTTP service used for standard-run placement
OPTIMIZER_METHOD	RG	GPUspb solver method passed in the request body
OPTIMIZER_VERBOSE	0	Per-call log verbosity (0–3)
PROFILING_CONFIGS_PER_JOB	2	How many distinct GPU configurations a single instance profiles before falling through to a standard run
EXECUTOR	docker	Executor backend for the in-process JobRunner: docker or mock-slurm. Workers always use the Docker CLI
IJM_DRIFT_HEARTBEAT_S	15	Period of the slot-tracker drift heartbeat that reconciles mem_used against db_used per node
WORKER_GPU_MODE	runtime	Worker-side; how Docker exposes GPUs to training containers (runtime / cdi / none)
IMAGE_TAG_OVERRIDE	unset	Worker-side; rewrites :latest to the given tag. Used on matemagician to pick the CUDA-10.1-compatible :legacy image
HOST_ROOT, HOST_PROJECT_ROOT	/host, ${PWD}/..	Resolve container-internal vs. host paths for Docker bind mounts

Table 3.4: Key environment variables.

4. End-to-End Scenarios

This section presents the cost model and the two end-to-end scenarios used to validate IJM on the two-node cluster. Scenario 1 stresses the single-type profile sweep and the deadline-driven URGENT migration; Scenario 2 mixes types so the A40 slots are held by tight-deadline `cnn_big` pins and the P600 slots by slack `lstm-small` patients. Both runs expose the same horizon-myopia behaviour in upstream GPUspb. Implementation notes accumulated while making the scenarios reproducible are collected in §4.5.

4.1 Objective and cost model

The scheduler minimises

$$\textsc{TotalCost} \;=\; \sum_{j} \texttt{GPUcost}(j) + \sum_{j} \texttt{TardWeight}(j) \cdot \max\bigl(0,\, \texttt{finish}(j) - \texttt{deadline}(j)\bigr),$$

where `GPUcost` is wall-clock × hourly rate (Table 4.1, left) and `TardWeight` scales with priority (right). Slack deadlines favour the cheapest bundle; deadline pressure shifts urgent jobs onto the fastest available.

GPU bundle	$/hour
`QuadroP600×1`	0.030
`QuadroP600×2`	0.055
`A40×1`	0.300
`A40×2`	0.550

Priority	TardWeight ($/sec)
1	0.000254
2	0.000301
3	0.000349
4	0.000396
5	0.000444

Table 4.1: Bundle hourly rates and per-priority tardiness weights.

The cluster used throughout this section is two profiling-capable nodes, four slots total: `matemagician` with 2×`QuadroP600` and `polimi-gpu` with 2×`A40`. Hourly rates are as in Table 4.1. For each new job type the profile sweep exercises all four `(node, GPU-count)` cells (`configs_per_job`=2, four instances).

4.1.1 Proxy vs. true objective

The cost above is the modelled objective. The C++ optimiser's per-decision proxy (`proxy_function_min` / `proxy_function_max` in GPUspb) is a cheaper surrogate.

In the minimisation form used by `method=RG`, tardiness is evaluated at the *next-event horizon* SIM_TIME=CURRENT_TIME+FIRST_FINISH_TIME rather than at each job's predicted completion. A placement on slow hardware therefore contributes zero in-proxy tardiness as long as the deadline is still in the future at that horizon.

The proxy also applies a 100× multiplier to the worst-case tardiness of jobs left *unscheduled* in a candidate plan, but that penalty only fires when a candidate plan exists in which a job ends up `sch.isEmpty()` (i.e. has no configuration left to place it on). Combined with Random Greedy's structure — jobs are placed in priority order onto *currently-free* capacity only, with no speculative preemption of incumbents — three consequences shape every URGENT placement we observed. First, when every candidate placement for the arrival has its deadline still in the future at the horizon, the proxy reads zero tardiness for each of them and picks the one with the lowest `vmCost` regardless of whether that placement is actually fast enough

to finish on time. Second, the 100× unscheduled penalty dominates any plan that leaves a prio-4-or-5 peer with no configuration, so when the cluster is oversubscribed at arrival time the optimiser will pick the plan with the fewest unscheduled high-priority peers — typically by routing the arrival to the slowest free bundle while keeping incumbents in place. Third, even when the cluster has spare capacity, RG only ever considers configurations whose slots are *already free*; the arrival cannot pre-empt an incumbent peer to take a faster slot because `Heuristic::assign` simply asks `system.assign_to_node(...)` for free capacity, gets none, and falls through to the slower bundle. Scenario 1 (§4.3) shows the first and third hitting together; Scenario 2 (§4.4) shows the second at work.

The proxy never names "preemption": it just penalises "unscheduled in this plan". The `proxy_function_max` variant (`method=PR`) uses per-job `sch.get_selectedTime()` instead and is deadline-aware, but it is not the default. We keep `method=RG` throughout to mirror the reference deployment. The behaviour described here is the one observed on the upstream GPUspb checkout used for this report.

4.2 Available job types

Type	Architecture	Dataset
`lstm-small`	LSTM 1-layer, 128 hidden	MNIST (sequence)
`lstm-big`	LSTM 3-layer, 256 hidden, dropout 0.2	MNIST (sequence)
`convnet`	3-layer CNN + BN	CIFAR-10
`efficientnet`	MBConv EfficientNet	CIFAR-10
`cnn_big`	10-block CNN, 3→ . . . →512 channels	synthetic 128×128×3

Table 4.2: Training containers under `runtime/`. All share the same checkpoint contract (legacy pre-1.6 `torch.save`, loaded with `strict=False`), so any job can resume across nodes regardless of the destination's torch version.

Type	A40×1 [s/ep]	A40×2 [s/ep]	QuadroP600×1 [s/ep]	QuadroP600×2 [s/ep]
`lstm-small`	3.56	4.81	5.47	10.93
`cnn_big`	3.118	**2.849**	46.376	**27.693**

Table 4.3: Steady-state per-epoch wall-clock from `profiling_results` (mean of post-warmup epochs). Bold marks 2-GPU bundles that beat their 1-GPU counterpart on wall-clock. `cnn_big` wins 1.09× on A40×2 and 1.68× on P600×2; it is the only type used by Scenario 1.

Type	A40×1 [$/ep]	A40×2 [$/ep]	P600×1 [$/ep]	P600×2 [$/ep]
`lstm-small`	0.000297	0.000735	0.0000456	0.000167
`cnn_big`	0.000260	0.000435	0.000386	0.000423

Table 4.4: GPUcost per epoch, computed as per-epoch[s] × hourly-rate (Table 4.1) / 3600. Multiply by `epochsTotal` to obtain the optimiser's energy term for a plan that runs the job to completion on the listed bundle.

4.3 Scenario 1 — single type

We submit five `cnn_big` jobs (Table 4.5). Patient deadlines (+30 min, +2 h) are deliberately loose: a plan that preempts a patient to free A40 for URGENT carries near-zero true cost — yet, as we shall see, the proxy never proposes one.

Label	Type	Priority	Deadline	Epochs	Submitted at
JOB1	`cnn_big`	4	+30 min	75	$t = 0$ s
JOB2	`cnn_big`	4	+30 min	75	$t = 7$ s
JOB3	`cnn_big`	2	+2 h	50	$t = 14$ s
JOB4	`cnn_big`	1	+2 h	25	$t = 19$ s
URGENT⋆	`cnn_big`	5	+10 min	200	t=7.5 min

Table 4.5: Scenario 1 submissions.

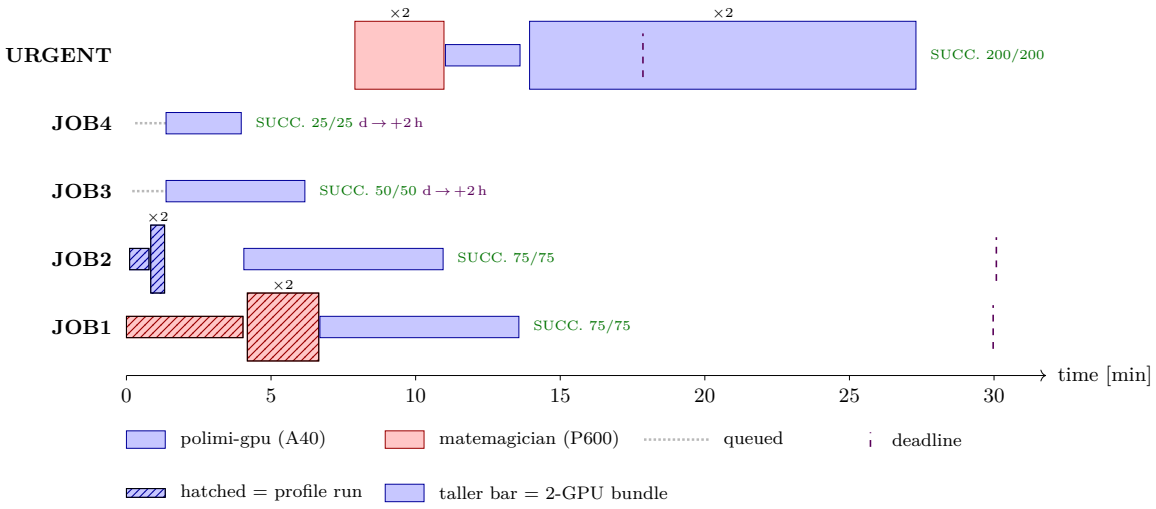

Figure 4.1: Scenario 1 timeline (run 2026-05-28, t=0 at first dispatch). URGENT lands on the free P600×2 even though A40 would meet its deadline, then migrates to A40×1 and upgrades to A40×2 as the prio-4 patients finish. The proxy mechanic behind this trajectory is unpacked in §4.3.3.

Bundle	per-epoch [s]	GPUcost/ep [$]	50-ep [$]	200-ep [$]	Best for
A40×1	3.118	0.000260	**0.0130**	0.0520	medium-deadline cnn_big
A40×2	2.849	0.000435	0.0217	0.0870	**≤10-min URGENT**
QuadroP600×1	46.376	0.000386	0.0193	0.0772	**slack runs (cheapest)**
QuadroP600×2	27.693	0.000423	0.0211	0.0846	— (dominated)

Table 4.6: Per-bundle data the optimiser sees for `cnn_big` after Scenario 1's profile sweep. Per-epoch is the steady-state mean from `profiling_results` (warmup epoch excluded). Full 200-epoch walls: A40×1 = 10.4 min, A40×2 = 9.5 min, P600×1 = 154.6 min, P600×2 = 92.3 min. Both A40 bundles fit URGENT's 10-minute deadline (with A40×1 just 24 s over); P600 overruns by more than an hour.

4.3.1 Decision timeline

t [min]	Job(s)	Decision (why)
0.0	J1, J2	J1 begins P600 sweep; J2 begins A40 sweep. "Profile-always" gate: neither is yet a standard candidate.
1.4	J3, J4	First OPT pass with enough profile coverage; J3 (prio 2) and J4 (prio 1) placed on A40×1 each (both A40 slots). J1, J2 still profile-owing ⇒ invisible to OPT.
3.0	J4	J4 ends naturally (25/25). A40 slot frees.
4.1	J2	J2 sweep finishes; OPT places J2 standard on the freed A40 slot.
6.2	J3	J3 ends naturally (50/50). A40 slot frees.
6.7	J1	J1 sweep finishes; OPT places J1 on the freed A40 slot. Cluster: J1, J2 on A40×1; P600 idle.
7.9	**URGENT**	**Submitted (prio 5, deadline +10 min). OPT picks P600×2 mate.** Both A40 held by prio-4 patients; RG only places on currently-free capacity (§4.3.3), so A40 plans are not generated; among free P600 configs the "cheapest if deadline reachable, else fastest" rule picks P600×2.
11.0	**URGENT**	**J2 ends naturally (75/75). OPT migrates URGENT → A40×1.** With URGENT now in currentSched, its proxy completion collapses from 92.3 min (P600×2) to 10.4 min (A40×1); migrate dominates kept-same.
13.6	**URGENT**	**J1 ends naturally (75/75). OPT upgrades URGENT → A40×2** (both A40 slots free; faster bundle is cheaper at horizon).
17.9	URGENT	Deadline (purple tick on Fig.4.1); URGENT still has ∼9 min remaining.
27.3	URGENT	URGENT ends (200/200) ∼10 min past deadline. w_5×600 s = ≈$0.27 tardiness incurred.

Table 4.7: Scenario 1 events. Migrations are triggered by patients finishing naturally, not by a policy decision to clear A40 for URGENT — the bug §4.3.3 dissects.

4.3.2 URGENT plan comparison — true objective

At URGENT's submission (t=7.9 min), the true objective would order the plans by full-job ExecTime + TardCost against URGENT's +10-min deadline; priority-5 tardiness is weighted at w_5=0.000444 \$/s.

Plan for URGENT	ExecTime [s]	Tardy [s]	GPUcost [$]	TardCost [$]	**TotalCost [$]**	True-cost rank
A40×1 (preempt 1)	624	24	0.0520	0.011	**0.063**	**1 (best)**
A40×2 (preempt 2)	570	0	0.0870	0.000	0.087	2
P600×2 (preempt 2)	5 539	4 939	0.0846	2.193	2.278	3
P600×1 (preempt 1)	9 275	8 675	0.0772	3.852	3.929	4

Table 4.8: True-objective ranking of URGENT plans at submission. ExecTime = 200×per-epoch from Table 4.6; Tardy = max(ExecTime−600, 0) against the 10-min deadline; TardCost = Tardy × w_5. Both A40 plans land within seconds of deadline (A40×1 is cheaper because it preempts only one patient); both P600 plans overshoot by more than an hour. This is *not* the cost the proxy used to decide; see §4.3.3.

4.3.3 Why URGENT lands on the slowest 2-GPU bundle

Table 4.8 ranks A40 plans first; the run picks P600×2. Three proxy mechanics combine to force this choice.

1. **Horizon-myopic tardiness.** `proxy_function_min` evaluates each job's tardiness at SIM_TIME=CURRENT_TIME+FIRST_FINISH_TIME, where `first_finish_time` is the soonest currently-scheduled finish. At URGENT's arrival, the A40 patients have ∼30 ep remaining on A40×1 (≈ 125 s); URGENT is 200 ep ahead on any bundle, so

it cannot finish in this horizon. Its deadline (+10 min= 600 s) is still in the future at SIM_TIME, so *every* URGENT placement gets in-proxy tardiness = 0. Tardiness drops out of the URGENT-on-X vs. URGENT-on-Y comparison.

2. **Random Greedy never proposes a plan that preempts an incumbent.** `Heuristic::assign` (the workhorse called by every greedy / random-greedy iteration) calls `system.assign_to_node(GPU_type, n_GPUs)` which only returns a node if a currently-free slot exists; it has no path that displaces a job already in `currentScheduling`. RG processes jobs in priority order: URGENT first. When URGENT asks for A40, both A40 slots are already taken by the prio-4 patients in currentSched, so `assign_to_node` returns empty and RG falls through to `assign_to_suboptimal`, which iterates configurations in cost-then-speed order using *only* currently-free capacity. The only configurations with free slots are P600×1 and P600×2. The "preempt the patient, place it on P600" counter-plan that would let URGENT have A40 is never even generated.
3. **P600×2 vs. P600×1: the deadline-missing fallback.** Between the two P600 configurations RG *does* actually consider, `Optimal_configurations::get_best_setup` applies the rule "if a configuration can meet the deadline, pick the cheapest; otherwise, pick the fastest". URGENT's 10-minute deadline cannot be met on either P600 bundle (P600×1 = 154.6 min wall, P600×2 = 92.3 min), so the rule's fallback branch fires and RG picks *fastest* = P600×2. P600×1 would have been cheaper per epoch, but the "cheapest" branch only runs when the deadline is reachable; here it is not, so cost is ignored.

Configuration considered by RG	Free at submit?	RG outcome	Why
A40×1 (single slot)	*no* (both A40s held by patients)	not considered	`assign_to_node` returns empty for occupie
A40×2 (both slots)	*no* (both A40s held by patients)	not considered	`assign_to_node` returns empty for occupie
P600×1	yes (both P600s free)	considered, rejected	deadline unreachable; "pick fastest" branc
P600×2	yes (both P600s free)	**chosen**	deadline unreachable; this is the fastest fre

Table 4.9: What Random Greedy actually evaluates at URGENT submission. A40 configurations never enter the comparison because `Heuristic::assign` has no path that preempts an incumbent in `currentScheduling`; the prio-4 patients hold both A40 slots and RG just sees "A40 unavailable". Among the P600 configurations that are free, the "cheapest if deadline reachable, else fastest" rule picks P600×2 (92.3 min wall vs. P600×1's 154.6 min, both far over URGENT's 10-min deadline).

Once URGENT is actually running on P600×2 it becomes part of `currentSched`. The next opt round at $t \approx 11.0$ sees JOB2 succeeded (its A40×1 slot freed) and `first_finish_time` now anchored on JOB1. Re-running the same proxy with URGENT already running on P600×2, migrating it to the now-free A40×1 is cheaper because URGENT's projected completion collapses from 92.3 min to 10.4 min; in-proxy URGENT tardiness drops accordingly and the GPU-rate increase is two orders of magnitude smaller in dollars. Two rounds later, when JOB1 also completes at $t \approx 13.6$, the second A40 frees and URGENT upgrades to A40×2.

`method=PR` (with `proxy_function_max`, deadline-aware per job) would pick A40×2 from the first round, but we keep `method=RG` as the reference deployment.

4.4 Scenario 2 — two types, forced 2-GPU placement

Scenario 2 introduces a second job type and constructs a workload where every available bundle has consequences. Two priority-5 `cnn_big` pins occupy the A40 slots; an urgent

`cnn_big` arrives with a deadline that admits P600×2 (18.5 min) but not P600×1 (31 min); the A40 slots are blocked by the pins. The proxy — as we shall see — still picks P600×1.

Label	Type	Priority	Deadline	Epochs
PIN-A	B (`cnn_big`)	5	+10 min	80
PIN-B	B (`cnn_big`)	5	+10 min	80
P-LSTM-1	S (`lstm-small`)	1	+8 h	200
P-LSTM-2	S (`lstm-small`)	1	+8 h	200
URGENT⋆	B (`cnn_big`)	5	+22 min	40

Table 4.10: Scenario 2 submissions. URGENT's +22-min deadline admits P600×2 (18.5 min) but not P600×1 (31 min); A40 is blocked by the priority-5 pins.

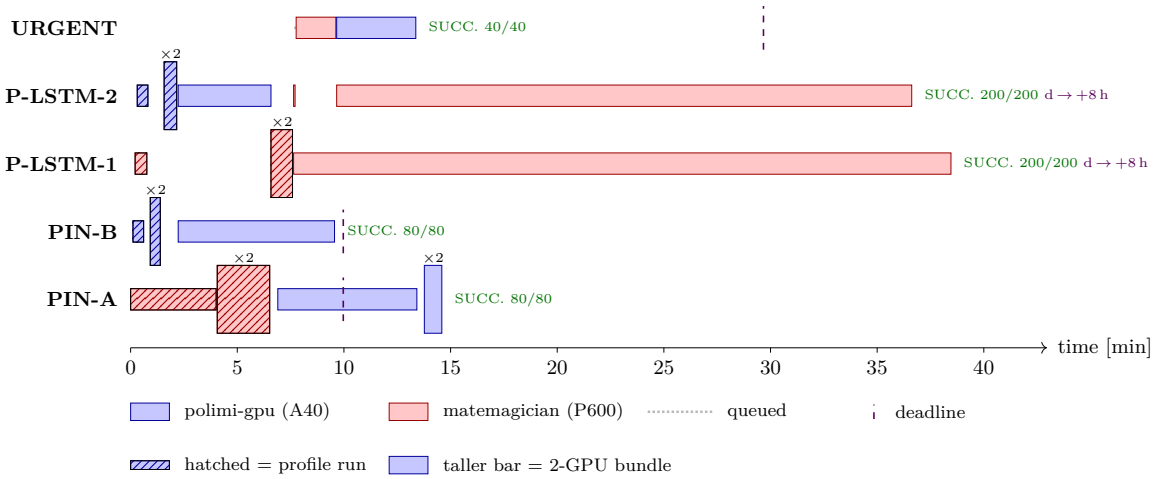

Figure 4.2: Scenario 2 timeline (run 2026-05-28, t=0 at first dispatch). URGENT lands on P600×1 (not P600×2) by preempting one lstm-small, then migrates to A40×1 when PIN-B ends.

4.4.1 URGENT plan comparison

Plan	ExecTime [s]	GPUcost [$]	Tardiness [s]	TardCost [$]	Total [$]	Decision
A40×1 (preempt 1 pin)	125	0.0104	0	0.0	≈1.56†	rejected (unsched pin)
A40×2 (preempt both pins)	114	0.0174	0	0.0	≈3.11†	rejected (2 unsched pins)
P600×2 (preempt both lstm-small)	1 108	0.0169	0	0.0	≈0.017	not chosen (myopia + slack)
P600×1 (preempt 1 lstm-small)	**1 855**	**0.0154**	0‡	0.0‡	**≈0.015**	✓ chosen

Table 4.11: URGENT plan comparison. ExecTime = 40 × per-epoch from Table 4.3; TardCost uses the priority-5 weight. † A40 plans look free of URGENT's tardiness, but each evicted prio-5 pin becomes ∼58 min tardy on its own +10-min deadline (≈ $1.55 per victim), so they are dominated. ‡ In proxy at horizon, URGENT's tardiness on P600×1 reads zero: the horizon (next-finish of a scheduled job) is well before URGENT's +22-min deadline, so the proxy is blind to the 31-min real ExecTime. P600×1 wins on vmCost-at-horizon over P600×2, which is the horizon-myopic bug from §4.3.3 re-appearing in the lstm-small-eviction context.

4.4.2 Decision timeline

t [min]	Job(s)	Decision (why)
0.0–6.6	PINs, P-LSTMs	Profile sweep over all eight (type, GPU-bundle) cells. PIN-A carries the P600 sweep; PIN-B + P-LSTM-2 carry the A40 sweep; P-LSTM-1 atomic-claims the remaining cells.
2.2	PIN-B, P-LSTM-2	Both finish their profile budgets; OPT places PIN-B on A40×1 polimi and P-LSTM-2 on A40×1 polimi (only profiled bundles available).
6.6	PIN-A, P-LSTM-2	PIN-A sweep finishes; OPT prefers prio-5 PIN-A over prio-1 P-LSTM-2 on A40 ⇒ drop-preempt P-LSTM-2; new placement: PIN-A on A40×1 polimi.
7.6	P-LSTM-1, P-LSTM-2	OPT places P-LSTM-1 and the just-evicted P-LSTM-2 on matemagician P600×1 each (cheapest bundle for lstm-small).
7.7	**URGENT**	**Submitted (prio 5, deadline +22 min). OPT places URGENT on P600×1 mate, drop-preempting P-LSTM-2.** A40 is held by PIN-A, PIN-B (both prio 5, tight deadlines) — RG refuses to displace them. Among free P600 configs the "cheapest if deadline reachable, else fastest" rule picks P600×1 because the horizon-myopic proxy reads URGENT's tardiness as zero (deadline still future at SIM_TIME) and P600×1 has the lower vmCost — the horizon-myopia bug from §4.3.3 resurfacing.
9.6	**URGENT**	**PIN-B ends naturally (80/80). OPT migrates URGENT → A40×1 polimi** (proxy now sees URGENT's real P600 finish time and prefers A40); P-LSTM-2 is replaced on P600×1 in the same round.
13.4	URGENT, PIN-A	URGENT ends (40/40) well inside its +22-min deadline; A40 slot frees. OPT upgrades PIN-A from A40×1 to A40×2 (faster bundle, both slots free).
13.8	PIN-A	Applied pending migrate PIN-A → A40×2; PIN-A ends ∼0.8 min later (already at 78/80).
∼36–38	P-LSTMs	Both lstm-small patients finish naturally on P600×1 each (their +8-h deadlines are far in the future).

Table 4.12: Scenario 2 events. URGENT's placement is the same horizon-myopia mechanism as Scenario 1, here biting on the choice between P600×1 and P600×2 rather than between P600 and A40 (the latter is blocked by the prio-5 pins).

4.5 Implementation notes

Five issues surfaced while bringing the scenarios to reproducible state; each is summarised below and detailed in the repo's `CLAUDE.md` / commit messages.

- **Stop/run reorder race.** A migration's `/stop` and `/run` could hit a worker out of order under `method=PR`, so the new container started before the previous CUDA context drained; `dispatch_with_slot` now awaits any in-flight `/stop` on the same node first.
- **Per-epoch progress lag.** Reader opened a fresh SSH-tunnelled PG connection per epoch (∼5 s each) and the backlog could overrun the 60 s drain timeout; reader now coalesces into a once-per-second background writer.
- **Profile-claim thrash.** A hot-path sweep deleted live claims during a transient `is_profiling_run=FALSE` window; replaced with lifecycle-event-driven cleanup plus a 15-min orphan GC in the 60-minute `_queue_watcher`.
- **Self-inflicted optimiser wakes.** Every slot-freed `NOTIFY` woke the optimiser, including notifies caused by the plan's own auto-preempts, so RG re-ran on partial state and thrashed. The payload now carries a reason; `auto_preempt` is swallowed (the plan's own dispatch picks up the freed slot). Migrate plans are staged

in `state.pending_migrates` and committed by the slot listener once the source `/stop` drains, so the swallow doesn't strand them.

- **Cross-version checkpoints.** Pre-1.6 `torch.save` + `strict=False` load lets a job migrate between `matemagician` (torch 1.5.1) and `polimi-gpu` (torch 2.6) without losing progress; Scenario 1 exercises both directions.

4.6 Reproducibility

Per-bundle costs are taken from `profiling_results.duration_seconds` (steady-state per-epoch mean, warmup excluded) and the hourly rates in Table 4.1. Timelines mirror per-instance state transitions from a 0.5 s API poller. Scenarios are driven by `infra/e2e_scenario.sh` and `infra/e2e_scenario_2types.sh`; slot metrics are exposed at `GET /admin/slots`, and API logs land in the `ijm-api` container (`docker logs ijm-api`) brought up by `infra/ijm up`.

5. Conclusion

5.1 Summary of contributions

IJM extends the ANDREAS prototype from a single-host scheduler into a distributed control plane: one FastAPI service drives per-node workers over SSH-tunnelled HTTP, and an external GPUspb optimiser is consumed as a peer service rather than embedded in the scheduler. On top of that topology the report names three substantive contributions. A *profile-always* policy keyed by the unique index on `(job_id, gpu_config)` — so that profile measurements are atomically shared across instances of the same job type. *Cross-version checkpoint resume* between PyTorch 1.5.1 (CUDA 10.1, `matemagician`) and PyTorch 2.6 (CUDA 12.4, `polimi-gpu`), enabling an in-flight job to migrate between heterogeneous GPU generations without losing progress. And an empirical characterisation of the horizon-myopia in GPUspb's `method=RG` proxy (Chapter 4), traced through both scenarios to three combining mechanics: horizon-myopic tardiness, `Heuristic::assign`'s refusal to displace incumbents, and the "cheapest if deadline reachable, else fastest" fallback.

5.2 Limitations

The reference deployment uses GPUspb's `method=RG`, whose proxy is horizon-myopic; the deadline-aware `method=PR` variant exists upstream but is not the default and has not been validated end to end against the IJM control plane. The report contains no baseline-algorithm comparison — IJM's behaviour is documented against the true objective, but not yet against FIFO, EDF, or priority-only baselines on the same workloads. Worker-side coverage is end-to-end only (`worker/tests/` is empty); only two GPU generations (A40 and QuadroP600) have been validated; and the cluster used throughout Chapter 4 is two nodes with four GPU slots in total, so the scaling properties of the four-phase pass at tens of nodes are an open question.

5.3 Future work

Three follow-ups have concrete entry points in the codebase. First, swap the Docker executor for SLURM: the `Executor` abstraction in `backend/src/job_runner.py` already isolates the Docker calls, and the worker's HTTP surface is small enough to back with a SLURM batch-submit shim. Second, re-run Scenarios 1 and 2 with `OPTIMIZER_METHOD=PR`: `proxy_function_max` is per-job deadline-aware, so the prediction from §4.3.3 is that URGENT lands on A40×2 in round 1 rather than migrating into it over two patient-completion events. Third, extend the two-node evaluation to a larger cluster once a third GPU class is available; this is where the profile-claim's atomicity and the optimiser-call latency (currently a 30 s timeout) will be stressed.

Bibliography

[AF+21] Danilo Ardagna, Federica Filippini, et al. ANDREAS – artificial intelligence training scheduler for accelerated clusters. Technical report, Politecnico di Milano / TETRAMAX, 2021. Horizon 2020 project deliverables D1–D3.